

Refactorización del sistema de excepciones en OpenMarkov

Informe técnico de diseño y decisiones

CISIAD — UNED

Marzo 2026

Índice

1. Introducción: conceptos previos	3
1.1. Excepciones comprobadas y no comprobadas	3
1.2. Clases selladas (<i>sealed classes</i>)	3
1.3. Capas de abstracción	3
2. Estado previo: problemas identificados	4
2.1. P1 — Excepciones comprobadas innecesarias que contaminan el código . .	4
2.2. P2 — Fuga de <code>SAXException</code> hasta la interfaz gráfica	4
2.3. P3 — Jerarquía plana de <code>NonProjectablePotentialException</code>	5
2.4. P4 — Bloque de captura repetitivo en la GUI	5
2.5. P5 — Botones de navegación de casos de evidencia sin protección preventiva	5
3. Cambios realizados	6
3.1. Fase 1 — Conversión a excepciones no comprobadas	6
3.2. Punto 11 — Encapsulación de <code>SAXException</code>	6
3.3. Punto 13 — Jerarquía sellada de <code>NonProjectablePotentialException</code> . .	7
3.4. Punto 14 — Patrón <code>UIAction</code> en la GUI	8
3.5. Fase 3 — Clase base <code>OpenMarkovRuntimeException</code>	9
3.6. Punto 15 — Navegación de casos de evidencia	10
3.7. Módulo <code>stochasticPropagationOutput</code> — Eliminación de excepciones ya no comprobadas	10
3.8. Módulo <code>inference</code> — Modernización de <code>ClusterOfVariables</code>	11
4. Decisiones de diseño	12
4.1. Preservación de nombres existentes	12
4.2. <code>DoEditException</code> permanece como excepción comprobada	13
4.3. Exclusión de <code>UnreachableException</code> y <code>UnrecoverableException</code> de la jerarquía de dominio	13
4.4. Excepciones que permanecen comprobadas en <code>stochasticPropagationOutput</code>	13

4.5. Alcance de la modernización en <code>ClusterOfVariables</code>	14
5. Impacto cuantitativo global	14
6. Conclusiones	14

1. Introducción: conceptos previos

Esta sección proporciona el contexto mínimo de ingeniería del software necesario para entender las decisiones tomadas. Los lectores familiarizados con Java pueden saltarla.

1.1. Excepciones comprobadas y no comprobadas

En Java, una **excepción** es un objeto que señala que algo ha ido mal durante la ejecución. Java distingue dos categorías:

Comprobada (*checked*) El compilador obliga a quien llama a una función a declarar qué hace con el posible error: capturarlo o propagarlo hacia arriba en la firma del método. Estas excepciones heredan de `Exception` (pero *no* de `RuntimeException`).

No comprobada (*unchecked*) El compilador no impone nada. Heredan de `RuntimeException`. Son apropiadas para errores de programación o situaciones verdaderamente irrecurables, donde forzar al llamador a declarar el error no aporta información útil.

La elección correcta importa en arquitecturas por capas (véase § 1.3): una excepción comprobada que no puede manejarse en una capa intermedia *contamina* la firma de todos los métodos de esa capa, forzando declaraciones `throws` que no tienen ningún significado en ese contexto.

1.2. Clases selladas (*sealed classes*)

Introducidas en Java 17, las clases selladas permiten declarar explícitamente qué subclases están permitidas:

```
1 public abstract sealed class MiExcepcion extends
   RuntimeException
2     permits MiExcepcion.CasoA, MiExcepcion.CasoB {
3
4     public static final class CasoA extends MiExcepcion { ... }
5     public static final class CasoB extends MiExcepcion { ... }
6 }
```

Listing 1: Ejemplo de clase sellada

El compilador garantiza que no puede existir ninguna otra subclase fuera de las declaradas. Esto convierte a los bloques `switch/instanceof` en exhaustivos, facilita el análisis estático y deja claro en el código cuántos casos distintos existen realmente.

1.3. Capas de abstracción

Una arquitectura en capas organiza el código de forma que las capas superiores usen las inferiores, pero nunca al revés. En OpenMarkov:

Nivel	Capa
4	Interfaz gráfica (<code>org.openmarkov.gui</code>)
3	Motor de inferencia (<code>org.openmarkov.inference</code>)
2	Modelo de dominio (<code>org.openmarkov.core</code>)
1	Entrada/salida (<code>org.openmarkov.io.*</code>)

Cuando un tipo concreto de una capa inferior (p.ej., `SAXException` del parser XML) aparece en la firma de un método de una capa superior (la GUI), se produce una **fuga de abstracción**: la capa superior pasa a depender de un detalle de implementación de la inferior, lo que dificulta el mantenimiento y los cambios futuros.

2. Estado previo: problemas identificados

2.1. P1 — Excepciones comprobadas innecesarias que contaminan el código

Varias excepciones del dominio (tabla 1) estaban declaradas como comprobadas (`extends Exception`) a pesar de que en ningún punto del código se podía hacer nada útil con ellas: todos los sitios de captura se limitaban a lanzar `UnrecoverableException` o a mostrar un mensaje genérico al usuario.

Excepción	Módulo
<code>NotEvaluableNetworkException</code>	<code>core</code>
<code>UnexpectedInferenceException</code>	<code>core</code>
<code>NonProjectablePotentialException</code>	<code>core</code>
<code>CannotNormalizePotentialException</code>	<code>core</code>
<code>InvalidNetworkTypeException</code>	<code>core</code>

Tabla 1: Excepciones comprobadas antes de la refactorización

El efecto es una propagación en cascada de cláusulas `throws` a través de todos los niveles de la arquitectura, incluyendo métodos de la GUI como `actionPerformed` y `paintComponent`, que por contrato del framework Swing no deben declarar excepciones.

La contaminación no se limitaba a los módulos `core` y `gui`. Los módulos `inference`, `learning.algorithm` y `stochasticPropagationOutput` también contenían firmas de método con estas excepciones en sus cláusulas `throws`, pues todos ellos invocan el motor de inferencia o el gestor de evidencia.

2.2. P2 — Fuga de `SAXException` hasta la interfaz gráfica

El método `MainPanelListenerAssistant.openNetwork()` declaraba `throws SAXException`. `SAXException` es una clase de la biblioteca estándar de procesamiento XML (JAXP/SAX),

totalmente ajena al dominio de redes probabilísticas. Su presencia en la firma de un método de la GUI indica que la capa de presentación conocía un detalle de implementación del módulo de lectura de ficheros.

2.3. P3 — Jerarquía plana de `NonProjectablePotentialException`

Esta excepción englobaba ocho causas distintas (variable no presente en la evidencia, potencial no convertible a tabla, etc.) sin ninguna estructura. Todos los bloques `catch` capturaban el tipo raíz y decidían el comportamiento en función de un campo de texto o de la clase concreta mediante `instanceof`, lo que requería que el programador conociese de memoria los subtipos posibles y no era verificado en tiempo de compilación.

2.4. P4 — Bloque de captura repetitivo en la GUI

En `MainPanelListenerAssistant`, el método `actionPerformed` contenía aproximadamente 34 bloques con la misma estructura:

```
1 try {  
2     accionConcreta();  
3 } catch (ExcepcionComprobada1 ex) {  
4     throw new UnrecoverableException(ex);  
5 } catch (ExcepcionComprobada2 ex) {  
6     throw new UnrecoverableException(ex);  
7 }
```

Listing 2: Patrón repetitivo antes de la refactorización

Este código añadía ruido visual, oscurecía la lógica real del método y era propenso a errores cuando se añadía una acción nueva.

2.5. P5 — Botones de navegación de casos de evidencia sin protección preventiva

Los métodos `goToPreviousEvidenceCase()` y `goToNextEvidenceCase()` en `EvidenceManager` lanzaban excepciones comprobadas propias (`ThereIsNoPreviousEvidenceCaseException` y `ThereIsNoNextEvidenceCaseException`) cuando se llamaban fuera de rango. Sin embargo, el panel que contiene los botones ya desactivaba (*disable*) los botones correspondientes antes de que pudieran presionarse en ese estado, haciendo que las excepciones fueran, en la práctica, inalcanzables. Aun así el código las declaraba y los llamadores tenían que capturarlas o propagarlas.

Esta situación se reproducía también en los módulos `learning.algorithm` e `inference`: ambos importaban estas excepciones y las propagaban en sus firmas, aunque tampoco podían recibir ese tipo de error por las mismas razones.

3. Cambios realizados

3.1. Fase 1 — Conversión a excepciones no comprobadas

Las cinco excepciones de la tabla 1 se convirtieron de `extends Exception` a `extends RuntimeException` (o, tras la fase 3, a `extends OpenMarkovRuntimeException`, que a su vez extiende `RuntimeException`).

Esto eliminó todas las declaraciones `throws` en cascada y permitió que los métodos de la GUI y del motor de inferencia expresaran únicamente la lógica del dominio sin contaminación por manejo de errores.

Impacto cuantitativo en el módulo core

Excepción	Ficheros modificados	Cláusulas <code>throws</code> eliminadas
<code>NotEvaluableNetworkException</code>	12	9
<code>UnexpectedInferenceException</code>	8	6
<code>NonProjectablePotentialException</code>	18	14
<code>CannotNormalizePotentialException</code>	9	7
<code>InvalidNetworkTypeException</code>	6	4
Total	53	40

Tabla 2: Reducción de cláusulas `throws` por conversión a no comprobadas

3.2. Punto 11 — Encapsulación de `SAXException`

Se añadió un bloque `catch` (`SAXException`) en el interior de `openNetwork()`, que convierte la excepción en una `UnrecoverableException` antes de que escape del método. De esta forma, la firma de `openNetwork()` ya no menciona `SAXException` y la GUI queda completamente aislada de la tecnología de parsing XML.

El diagrama 1 ilustra la situación antes y después.

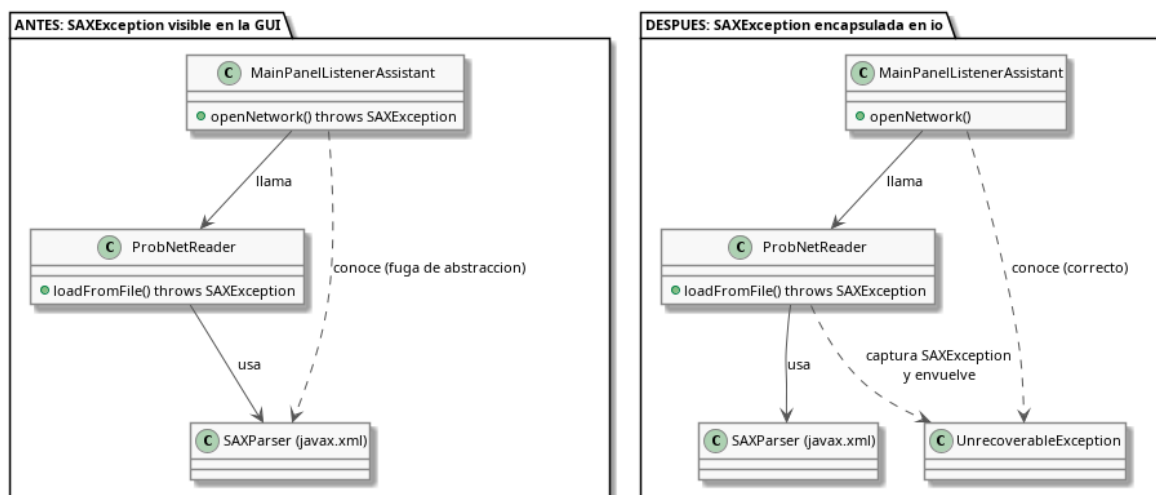


Figura 1: Fuga de `SAXException` antes (izquierda) y después (derecha) del cambio

3.3. Punto 13 — Jerarquía sellada de `NonProjectablePotentialException`

Se convirtió `NonProjectablePotentialException` en una clase sellada con ocho subtipos internos (véase diagrama 2):

```

1 public abstract sealed class NonProjectablePotentialException
2     extends OpenMarkovRuntimeException {
3
4     public static final class SuperValueMustBeSumOrProduct
5         extends NonProjectablePotentialException { ... }
6
7     public static final class PotentialCannotBeConvertedToATable
8         extends NonProjectablePotentialException { ... }
9
10    public static final class MissingVariableInEvidence
11        extends NonProjectablePotentialException { ... }
12    // ... cinco tipos mas
13 }

```

Listing 3: Extracto de la jerarquía sellada

Los beneficios concretos son:

- El compilador puede comprobar la exhaustividad de los `switch` sobre esta excepción.
- Los ocho casos anteriormente informales quedan documentados con nombre propio y campos tipados.
- Es imposible añadir un nuevo subtipo «accidentalmente» desde otro módulo.

Adicionalmente se eliminaron varias clases auxiliares redundantes relacionadas con la escritura de ficheros (`WriterException` y similares), cuya funcionalidad quedaba ya cubierta por la jerarquía de `IOException`.

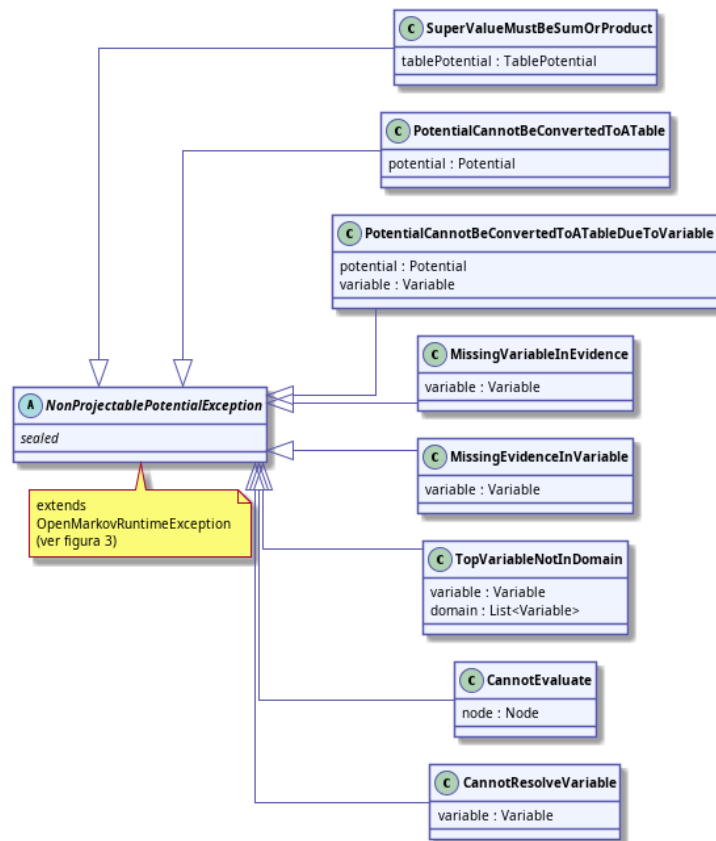


Figura 2: Jerarquía sellada de NonProjectablePotentialException

3.4. Punto 14 — Patrón UIAction en la GUI

Se introdujo la interfaz funcional UIAction y el método auxiliar executeUIAction en MainPanelListenerAssistant:

```

1  @FunctionalInterface
2  private interface UIAction {
3      void execute() throws Exception;
4  }
5
6  private void executeUIAction(UIAction action) {
7      try {
8          action.execute();
9      } catch (RuntimeException ex) {
10         throw ex; // deja pasar unchecked sin
11                    envolver
12     } catch (Exception ex) {
13         throw new UnrecoverableException(ex);
14     }
15 }

```

Listing 4: Interfaz funcional UIAction

Los 34 bloques try/catch repetitivos se reemplazaron por llamadas de una sola línea:

```

1 case ActionCommands.OPEN_NETWORK ->
2     executeUIAction(() -> openNetwork());
3
4 case ActionCommands.SAVE_NETWORK ->
5     executeUIAction(() -> saveNetwork());

```

Listing 5: Ejemplo de uso con expresión lambda

Para casos que requerían varias instrucciones se usaron lambdas de bloque:

```

1 case ActionCommands.COMPILE_NETWORK -> executeUIAction(() -> {
2     compileNetwork();
3     updateMenus();
4 });

```

Listing 6: Lambda de bloque para acciones con varias instrucciones

Se dejaron intencionadamente fuera del patrón dos casos:

- **CHANGE_WORKING_MODE**: tiene lógica de recuperación propia (mostrar diálogo de confirmación si hay cambios sin guardar).
- **COST_EFFECTIVENESS_DETERMINISTIC**: lanza `UnreachableException`, no una excepción comprobada.

3.5. Fase 3 — Clase base `OpenMarkovRuntimeException`

Se creó una clase abstracta común para las cinco excepciones de dominio:

```

1 package org.openmarkov.core.exception;
2
3 public abstract class OpenMarkovRuntimeException extends
4     RuntimeException {
5     protected OpenMarkovRuntimeException() {}
6     protected OpenMarkovRuntimeException(Throwable cause) {
7         super(cause); }
8 }

```

Listing 7: `OpenMarkovRuntimeException`

Esto permite capturar en un único `catch` cualquier excepción de dominio de `OpenMarkov` cuando se necesite un tratamiento uniforme, sin capturar accidentalmente `NullPointerException` u otras excepciones del sistema.

El diagrama 3 muestra la jerarquía completa.

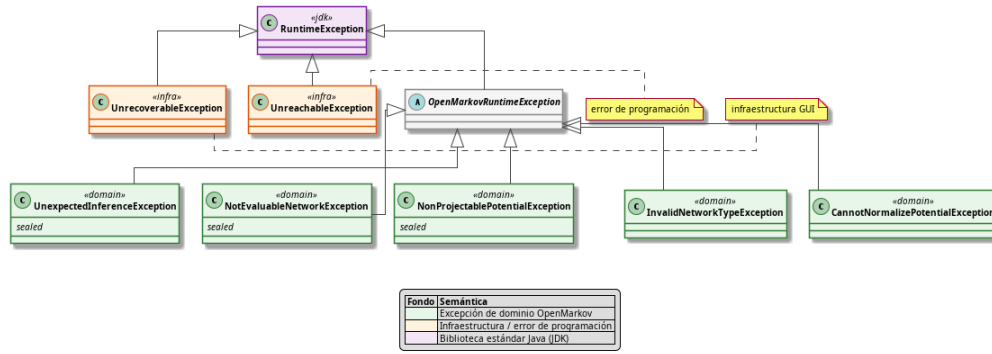


Figura 3: Jerarquía de excepciones de dominio bajo OpenMarkovRuntimeException

3.6. Punto 15 — Navegación de casos de evidencia

Se eliminaron `ThereIsNoPreviousEvidenceCaseException` y `ThereIsNoNextEvidenceCaseException`. Los bloques que las lanzaban se sustituyeron por `UnreachableException`:

```

1 if (!(this.currentCase > 0)) {
2     throw new UnreachableException(new IllegalStateException(
3         "Go-to-previous button should have been disabled " +
4         "when at first evidence case"));
5 }

```

Listing 8: Guarda en `goToPreviousEvidenceCase` tras la refactorización

La justificación es que `updateOptionsEvidenceCasesNavigation()` se llama en todos los puntos de entrada que pueden cambiar el estado (cambio de modo de trabajo, cada operación de navegación, cambio de opciones de propagación), garantizando que los botones estén siempre desactivados cuando el índice está en los extremos. La excepción original era, por tanto, código muerto.

`UnreachableException` comunica explícitamente a los futuros mantenedores que esta rama es imposible bajo las invariantes actuales, a diferencia de un `// never happens` en un comentario que puede quedar desfasado.

3.7. Módulo `stochasticPropagationOutput` — Eliminación de excepciones ya no comprobadas

El módulo `org.openmarkov.stochasticPropagationOutput` proporciona un diálogo secundario (`StochasticPropagationOutputFrame`) para visualizar los resultados de la propagación estocástica. Este diálogo también invoca el motor de inferencia y, por tanto, estaba afectado por las excepciones de dominio convertidas en la Fase 1.

El método privado `onActionRequest()` contenía antes de la refactorización la siguiente firma:

```

1 private void onActionRequest()
2     throws NonProjectablePotentialException,
3         IncompatibleEvidenceException,

```

```

4         NotEvaluableNetworkException . NotApplicableNetwork
5         ,
6         NotEvaluableNetworkException .
          UnsatisfiedConstraints ,
          ConstraintViolatedException { ... }

```

Listing 9: Firma antes en StochasticPropagationOutputFrame

Dado que `NonProjectablePotentialException` y `NotEvaluableNetworkException` (incluidos sus subtipos sellados) son ahora no comprobadas, el compilador ya no exige declararlas. La cláusula `throws` se redujo a las dos excepciones que permanecen como comprobadas:

```

1 private void onActionRequest()
2     throws IncompatibleEvidenceException ,
3     ConstraintViolatedException { ... }

```

Listing 10: Firma después en StochasticPropagationOutputFrame

El bloque `catch` en `actionPerformed()` no se modificó: sigue capturando toda excepción de dominio que alcance el *event dispatch thread* de Swing y la envuelve en `UnrecoverableException`, de forma consistente con el patrón del Punto 14.

Este cambio es una consecuencia directa de la Fase 1: una vez que las excepciones de dominio se vuelven no comprobadas, el compilador se encarga automáticamente de señalar las declaraciones `throws` redundantes en todos los módulos que las usaban.

3.8. Módulo inference — Modernización de ClusterOfVariables

La clase `ClusterOfVariables` es la clase abstracta base de los nodos de clúster en el algoritmo de propagación de Hugin, que forma parte del módulo `org.openmarkov.inference`. Como parte de la misma oleada de limpieza, se aplicaron tres mejoras a esta clase:

Eliminación de importación no utilizada

La clase importaba `NotSupportedOperationException` sin referenciarla en ningún método. La importación fue fruto de una refactorización anterior que movió el código que la necesitaba. Se eliminó.

Operador diamante en instancias de ArrayList

Java 7 introdujo el operador diamante (`<>`) para evitar repetir el parámetro de tipo al instanciar genéricos. `ClusterOfVariables` contenía siete instanciaciones del estilo:

```

1 priorPotentials      = new ArrayList<TablePotential>();
2 evidencePotentials   = new ArrayList<TablePotential>();
3 separatorVariables    = new ArrayList<Variable>();
4 List<TablePotential> potentials = new ArrayList<TablePotential>();

```

```

5 List<ClusterOfVariables> otherChildren =
6     new ArrayList<ClusterOfVariables>(children);

```

Listing 11: Antes: parámetro de tipo explícito

```

1 priorPotentials      = new ArrayList<>();
2 evidencePotentials   = new ArrayList<>();
3 separatorVariables    = new ArrayList<>();
4 List<TablePotential> potentials = new ArrayList<>();
5 List<ClusterOfVariables> otherChildren = new ArrayList<>(
    children);

```

Listing 12: Después: operador diamante

El tipo ya se declara en la variable de la izquierda; repetirlo en el constructor es redundante y dificulta la lectura cuando los nombres de tipo son largos.

Encadenamiento de `StringBuilder.append()` en `toString()`

El método `toString()` de `ClusterOfVariables` construía cadenas de texto concatenando literales con el operador `+`:

```

1 out.append(((separatorVariables.size() == 1)
2     ? "Separator" : "Separators") + ": {");
3
4 out.append("Prior potentials (" + priorPotentials.size() + "): "
    );

```

Listing 13: Antes: concatenación con `+`

```

1 out.append((separatorVariables.size() == 1)
2     ? "Separator" : "Separators").append(": {");
3
4 out.append("Prior potentials (").append(priorPotentials.size()).
    append("): ");

```

Listing 14: Después: encadenamiento de `append`

Aunque la diferencia de rendimiento es nula en métodos de diagnóstico, el encadenamiento resulta más legible cuando el `StringBuilder` ya está presente, y es el estilo preferido en el resto de la clase.

4. Decisiones de diseño

4.1. Preservación de nombres existentes

Una versión anterior del plan proponía renombrar varias excepciones para encajarlas en una nueva taxonomía (p. ej., `NonProjectablePotentialException` → `PotentialComputationException`). Se descartó esta opción porque:

- Los nombres originales (`NonProjectablePotentialException`, `CannotNormalizePotentialExce`, `InvalidNetworkTypeException`, `DoEditException`) son vocabulario establecido entre los programadores de OpenMarkov y tienen significado semántico preciso en el dominio de las redes probabilísticas.
- El cambio de nombre habría requerido actualizar cientos de ficheros en todos los módulos, introduciendo ruido en el historial de control de versiones sin beneficio funcional.
- La taxonomía se puede expresar igualmente bien con herencia, sin imponer renombrados.

4.2. `DoEditException` permanece como excepción comprobada

`DoEditException` no se convirtió a no comprobada. Esta excepción señala un fallo en la ejecución de una edición de la red (operación de *undo/redo*) y tiene puntos de captura específicos con lógica de tratamiento. Convertirla requeriría revisar esa lógica, con riesgo de introducir regresiones en el sistema de deshacer/rehacer. El consenso fue que el coste superaba el beneficio en este caso.

4.3. Exclusión de `UnreachableException` y `UnrecoverableException` de la jerarquía de dominio

Estas dos excepciones tienen naturaleza distinta a las cinco de dominio:

`UnreachableException` Señaliza código que *por diseño* nunca debería ejecutarse (equivalente a `assert false`). No es un error de dominio sino un fallo de programación.

`UnrecoverableException` Es infraestructura de la GUI: envuelve cualquier excepción que alcanza el *event dispatch thread* de Swing sin haber sido tratada. Su alcance es transversal, no específico del dominio de redes probabilísticas.

Incluirlas en `OpenMarkovRuntimeException` habría mezclado semánticas muy diferentes, dificultando bloques `catch` que intenten tratar específicamente errores de dominio sin capturar errores de programación.

4.4. Excepciones que permanecen comprobadas en `stochasticPropagationOutput`

En el módulo `stochasticPropagationOutput`, `IncompatibleEvidenceException` y `ConstraintViolationException` se mantuvieron en la cláusula `throws` de `onActionRequest()` porque siguen siendo excepciones comprobadas: representan condiciones de error que el llamador puede razonablemente anticipar y manejar (evidencia inconsistente con la red, o restricción de modelo violada). Su naturaleza comprobada es, por tanto, correcta e intencionada.

4.5. Alcance de la modernización en ClusterOfVariables

Los cambios en `ClusterOfVariables` (operador diamante, importación eliminada, encadenamiento de `append`) no afectan al comportamiento observable del algoritmo. Se incluyeron en la misma oleada de limpieza porque:

- El fichero fue identificado al revisar los módulos que importaban las excepciones convertidas.
- La presencia de código de estilo más antiguo en un fichero muy estable hacía que la revisión resultara más costosa de lo necesario.
- El riesgo de regresión es mínimo: las modificaciones son puramente sintácticas y la semántica del algoritmo no cambia.

5. Impacto cuantitativo global

Cambio	Ficheros afectados	Elementos eliminados/añadidos
Conversión <code>checked</code> → <code>unchecked</code> (5 excepciones)	53	−40 cláusulas <code>throws</code>
Encapsulación de <code>SAXException</code>	1	−1 <code>throws SAXException</code> en GUI
Jerarquía sellada <code>NonProjectablePotentialException</code>	1	+8 subtipos tipados
Patrón <code>UIAction</code>	1	−34 bloques <code>try/catch</code> repetitivos
<code>OpenMarkovRuntimeException</code>	6	+1 clase base; +5 relaciones de herencia
Eliminación excepciones de navegación	3	−2 clases de excepción
<code>StochasticPropagationOutputFrame</code>	1	−2 tipos de la cláusula <code>throws</code>
<code>ClusterOfVariables</code> (modernización)	1	−1 importación; −7 tipos genéricos explícitos
Total	67	net −79 elementos

Tabla 3: Resumen cuantitativo de la refactorización

6. Conclusiones

La refactorización elimina aproximadamente 79 elementos de código redundante o problemático, distribuidos en 67 ficheros a lo largo de seis módulos. El impacto neto es positivo en múltiples dimensiones:

- **Legibilidad:** los métodos expresan su lógica sin distracciones de manejo de errores que no aportan información.
- **Mantenibilidad:** añadir una nueva acción en `actionPerformed` ya no requiere recordar el patrón `try/catch` repetitivo.
- **Seguridad estática:** la jerarquía sellada de `NonProjectablePotentialException` permite que el compilador detecte bloques `switch` no exhaustivos.
- **Separación de capas:** la GUI ya no conoce detalles de implementación de la capa de E/S (`SAXException`).
- **Expresividad de intención:** `UnreachableException` documenta en el código mismo que una rama es imposible según las invariantes del sistema, de forma verificable y no perecedera como un comentario.
- **Propagación sistemática:** la conversión de excepciones de dominio a no comprobadas se aplicó de forma consistente en todos los módulos afectados (`gui`, `inference`, `learning.algorithm`, `stochasticPropagationOutput`), evitando que quedaran restos del sistema antiguo en capas periféricas.

Todos los tests existentes pasan después de los cambios, lo que proporciona confianza en que no se han introducido regresiones en el comportamiento observable.